



Python Strings Revisited

Strings

Traversing a String: A string can be traversed by accessing character(s) from one index to another. For example, the following program uses indexing to traverse a string from first character to the last.

Example:

```
message = 'Hello!'
```

```
index = 0
```

```
for i in message:
```

```
    print('message[' + index + '] = ' + i)
```

```
    index = index + 1
```

OUTPUT:

```
message[ 0 ] = H
```

```
message[ 1 ] = e
```

```
message[ 2 ] = l
```

```
message[ 3 ] = l
```

```
message[ 4 ] = o
```

```
message[ 5 ] = !
```

Concatenating, Appending and Multiplying Strings

Examples:

```
str1 = "Hello "  
str2 = "World"  
str3 = str1 + str2  
print("The concatenated string is : ", str3)
```

OUTPUT

The concatenated string is : Hello World

```
str = "Hello"  
print(str * 3)
```

OUTPUT

Hello Hello Hello

Concatenating, Appending and Multiplying Strings

Examples:

```
str = "Hello, "  
name = input("\n Enter your name : ")  
str += name  
str += ". Welcome to Python Programming."  
print(str)
```

OUTPUT

```
Enter your name : Arnav  
Hello, Arnav. Welcome to Python Programming.
```



Strings are Immutable

Python strings are immutable which means that once created they cannot be changed.

Whenever you try to modify an existing string variable, a new string is created.

String Formatting Operator

The % operator takes a format string on the left (that has %d, %s, etc) and the corresponding values in a tuple on the right. The format operator, % allow users to construct strings, replacing parts of the strings with the data stored in variables. The syntax for the string formatting operation is:

"<Format>" % (<Values>)

Example:

```
name = "Aarish"  
age = 8  
print("Name = %s and Age = %d" %(name, age))  
print("Name = %s and Age = %d" %("Anika", 6))
```

OUTPUT

```
Name = Aarish and Age = 8  
Name = Anika and Age = 6
```

Built-in String Methods and Functions

Function	Usage	Example
<code>capitalize()</code>	This function is used to capitalize first letter of the string.	<pre>str = "hello" print(str.capitalize())</pre> OUTPUT Hello
<code>center(width, fillchar)</code>	Returns a string with the original string centered to a total of <code>width</code> columns and filled with <code>fillchar</code> in columns that do not have characters.	<pre>str = "hello" print(str.center(10, '*'))</pre> OUTPUT **hello**

Built-in String Methods and Functions

`count(str, beg, end)`

Counts number of times `str` occurs in a string. You can specify `beg` as 0 and `end` as the length of the message to search the entire string or use any other value to just search a part of the string.

```
str = "he"
message = "helloworldhellohello"
print(message.count(str, 0, len(message)))
```

OUTPUT

3

`endswith(suffix, beg, end)`

Checks if string ends with suffix; returns True if so and False otherwise. You can either set `beg = 0` and `end` equal to the length of the message to search entire string or use any other value to search a part of it.

```
message = "She is my best friend"
print(message.endswith("end", 0, len(message)))
```

OUTPUT

True

Built-in String Methods and Functions

<code>find(str, beg, end)</code>	Checks if <code>str</code> is present in string. If found it returns the position at which <code>str</code> occurs in string, otherwise returns <code>-1</code> . You can either set <code>beg = 0</code> and <code>end</code> equal to the length of the message to search entire string or use any other value to search a part of it.	<pre>message = "She is my best friend" print(message.find("my", 0, len(message)))</pre> OUTPUT 7
<code>index(str, beg, end)</code>	Same as <code>find</code> but raises an exception if <code>str</code> is not found.	<pre>message = "She is my best friend" print(message.index("mine", 0, len(message)))</pre> OUTPUT ValueError: substring not found
<code>rfind(str, beg, end)</code>	Same as <code>find</code> but starts searching from the end.	<pre>str = "Is this your bag?" print(str.rfind("is", 0, len(str)))</pre> OUTPUT 5
<code>rindex(str, beg, end)</code>	Same as <code>rindex</code> but start searching from the end and raises an exception if <code>str</code> is not found.	<pre>str = "Is this your bag?" print(str.rindex("you", 0, len(str)))</pre> OUTPUT 8

Built-in String Methods and Functions

<code>isalnum()</code>	Returns True if string has at least 1 character and every character is either a number or an alphabet and False otherwise.	<pre>message = "JamesBond007" print(message.isalnum())</pre> OUTPUT True
<code>isalpha()</code>	Returns True if string has at least 1 character and every character is an alphabet and False otherwise.	<pre>message = "JamesBond007" print(message.isalpha())</pre> OUTPUT False
<code>isdigit()</code>	Returns True if string contains only digits and False otherwise.	<pre>message = "007" print(message.isdigit())</pre> OUTPUT True
<code>islower()</code>	Returns True if string has at least 1 character and every character is a lowercase alphabet and False otherwise.	<pre>message = "Hello" print(message.islower())</pre> OUTPUT False
<code>isspace()</code>	Returns True if string contains only whitespace characters and False otherwise.	<pre>message = " " print(message.isspace())</pre> OUTPUT True
<code>isupper()</code>	Returns True if string has at least 1 character and every character is an upper case alphabet and False otherwise.	<pre>message = "HELLO" print(message.isupper())</pre> OUTPUT True
<code>len(string)</code>	Returns the length of the string.	<pre>str = "Hello" print(len(str))</pre> OUTPUT 5
<code>ljust(width[, fillchar])</code>	Returns a string left-justified to a total of width columns. Columns without characters are padded with the character specified in the <code>fillchar</code> argument.	<pre>str = "Hello" print(str.ljust(10, '*'))</pre> OUTPUT Hello*****
<code>rjust(width[, fillchar])</code>	Returns a string right-justified to a total of width columns. Columns without characters are padded with the character specified in the <code>fillchar</code> argument.	<pre>str = "Hello" print(str.rjust(10, '*'))</pre> OUTPUT *****Hello

Slice Operation

A substring of a string is called a *slice*. The slice operation is used to refer to sub-parts of sequences and strings. You can take subset of string from original string by using `[ ]` operator also known as *slicing operator*.

Index from the start	P	Y	T	H	O	N	Index from the end
	0	1	2	3	4	5	
	-6	-5	-4	-3	-2	-1	

Examples:

```
str = "PYTHON"
print("str[1:5] = ", str[1:5])    #characters starting at index 1 and extending up
to but not including index 5
print("str[:6] = ", str[:6])      # defaults to the start of the string
print("str[1:] = ", str[1:])      # defaults to the end of the string
print("str[:] = ", str[:])        # defaults to the entire string
print("str[1:20] = ", str[1:20])  # an index that is too big is truncated down to
length of the string
```

OUTPUT

```
str[1:5] =  YTHO
str[:6] =  PYTHON
str[1:] =  YTHON
str[:] =  PYTHON
str[1:20] =  YTHON
```

Programming Tip: Python does not have any separate data type for characters. They are represented as a single character string.

Specifying Stride while Slicing Strings

In the slice operation, you can specify a third argument as the **stride**, which refers to the number of characters to move forward after the first character is retrieved from the string. By default the value of stride is 1, so in all the above examples where he had not specified the stride, it used the value of 1 which means that every character between two index numbers is retrieved.

Examples:

```
str = "Welcome to the world of Python"
print("str[2:10] = ", str[2:10])    # default stride is 1
print("str[2:10:1] = ", str[2:10:1]) # same as stride = 1
print("str[2:10:2] = ", str[2:10:2]) # skips every alternate character
print("str[2:13:4] = ", str[2:13:4]) # skips every fourth character
```

OUTPUT

```
str[2:10] = lcome to
str[2:10:1] = lcome to
str[2:10:2] = loet
str[2:13:4] = le
```

ord() and chr() Functions

ord() function returns the **ASCII** code of the character and **chr()** function returns character represented by a **ASCII** number. Examples:

<pre>ch = 'R' print(ord(ch))</pre> OUTPUT 82	<pre>print(chr(82))</pre> OUTPUT R	<pre>print(chr(112))</pre> OUTPUT p	<pre>print(ord('p'))</pre> OUTPUT 112
--	--	---	---

in and not in Operators

in and not in operators can be used with strings to determine whether a string is present in another string. Therefore, the in and not in operator are also known as membership operators.

Examples:

```
str1 = "Welcome to the world of Python
!!!"
str2 = "the"
if str2 in str1:
    print("Found")
else:
    print("Not Found")
```

OUTPUT
Found

```
str1 = "This is a very good book"
str2 = "best"
if str2 not in str1:
    print("The book is very good but it
may not be the best one.")
else:
    print ("It is the best book.")
```

OUTPUT
The book is very good but it may not be
the best one.

Comparing Strings

Operator	Description	Example
<code>==</code>	If two strings are equal, it returns True.	<pre>>>> "AbC" == "AbC" True</pre>
<code>!=</code> or <code><></code>	If two strings are not equal, it returns True.	<pre>>>> "AbC" != "Abc" True >>> "abc" <> "ABC" True</pre>
<code>></code>	If the first string is greater than the second, it returns True.	<pre>>>> "abc" > "Abc" True</pre>
<code><</code>	If the second string is greater than the first, it returns True.	<pre>>>> "abC" < "abc" True</pre>
<code>>=</code>	If the first string is greater than or equal to the second, it returns True.	<pre>>>> "aBC" >= "ABC" True</pre>
<code><=</code>	If the second string is greater than or equal to the first, it returns True.	<pre>>>> "ABc" <= "ABc" True</pre>

Iterating String

String is a sequence type (sequence of characters). You can iterate through the string using for loop.

Examples:

```
str = "Welcome to Python"
for i in str:
    print(i, end=' ')
```

OUTPUT

W e l c o m e t o P y t h o n

```
message = " Welcome to Python "
index = 0
while index < len(message):
    letter = message[index]
    print(letter, end=' ')
    index += 1
```

OUTPUT

W e l c o m e t o P y t h o n

The String Module

The string module consist of a number of useful constants, classes and functions (some of which are deprecated). These functions are used to manipulate strings.

Examples:

```
str = "Welcome to the world of Python"
print("Uppercase - ", str.upper())
print("Lowercase - ", str.lower())
print("Split - ", str.split())
print("Join - ", '-'.join(str.split()))
print("Replace - ", str.replace("Python", "Java"))
print("Count of o - ", str.count('o') )
print("Find of - ", str.find("of"))
```

OUTPUT

```
Uppercase -  WELCOME TO THE WORLD OF PYTHON
Lowercase -  welcome to the world of python
Split -  ['Welcome', 'to', 'the', 'world', 'of', 'Python']
Join -  Welcome-to-the-world-of-Python
Replace -  Welcome to the world of Java
Count of o -  5
Find of -  21
```

Programming Tip: A method is called by appending its name to the variable name using the period as a delimiter.

Working with Constants in String Module

You can use the constants defined in the string module along with the find function to classify characters. For example, if `find(lowercase, ch)` returns a value except `-1`, then it means that `ch` must be a lowercase character. An alternate way to do the same job is to use the `in` operator or even the comparison operation.

Examples:

```
# First Way
import string
print(string.find(string.
lowercase, 'g') != -1)
```

OUTPUT

True

```
# Second Way
import string
print('g' in string.
lowercase)
```

OUTPUT

True

```
# Third Way
import string
ch = 'g'
print('a' <= ch <= 'z')
```

OUTPUT

True